

RESEARCH ARTICLE

WILEY

Constraint programming approaches for the discretizable molecular distance geometry problem

Maira MacNeil¹ | Merve Bodur²

Department of Mechanical and Industrial Engineering, University of Toronto, Toronto, Ontario, Canada

Correspondence

Maira MacNeil, Department of Mechanical and Industrial Engineering, University of Toronto, Toronto ON M5S, Canada.
Email: m.macneil@mail.utoronto.ca

Abstract

The Distance Geometry Problem (DGP) seeks to find positions for a set of points in geometric space when some distances between pairs of these points are known. The so-called discretization assumptions allow us to discretize the search space of DGP instances. In this paper, we focus on a key subclass of DGP, namely the Discretizable Molecular DGP, and study its associated graph vertex ordering problem, the Contiguous Trilateration Ordering Problem (CTOP), which helps solve DGP. We propose the first constraint programming formulations for CTOP, as well as a set of checks for proving infeasibility, domain reduction techniques, symmetry breaking constraints, and valid inequalities. Our computational results on random and pseudo-protein instances indicate that our formulations outperform the state-of-the-art integer programming formulations.

KEYWORDS

combinatorial optimization, constraint programming, contiguous trilateration ordering, discretizable molecular distance geometry, discretization order

1 | INTRODUCTION

In its essence, Distance Geometry seeks to find positions for a set of points in geometric space when some distances between pairs of these points are known [8, 17]. This has many applications, including in molecular geometry, where Nuclear Magnetic Resonance (NMR) spectroscopy gives estimates of some interatomic distances and the three-dimensional structure must be determined [8]. Here, the points to be positioned are the atoms of a molecule. As seen in Figure 1(A), we have a set of five atoms and know some pairwise distances between them. We would like to give the atoms coordinates in Euclidean space so that the distances are preserved; a possible three-dimensional mapping of these atoms is pictured in Figure 1(B). In wireless sensor localization, the positions of wireless sensors such as smartphones must be determined using the estimated distance between sensors, but there is also a fixed component of the network such as routers [13]. Other applications include astronomy, robotics, statics and graph rigidity, graph drawing, and clock synchronization [8, 13, 17]. More recent applications have arisen from a new variant of the Distance Geometry Problem (DGP), namely dynamical DGP, including air traffic control, crowd simulation, multirobot formation, and human motion retargeting [17].

The input to the DGP can be represented as a graph, say G , where the vertices are the points we would like to position and weighted edges represent known distances between two points. Formally, we give the definition from [8].

Definition 1 (Distance geometry problem). Given an integer $K > 0$, and a simple, undirected graph $G = (\mathcal{V}, \mathcal{E})$ with edge weights $w : \mathcal{E} \rightarrow (0, \infty)$, find a function $x : \mathcal{V} \rightarrow \mathbb{R}^K$ such that for all $\{u, v\} \in \mathcal{E}$:

$$\|x(u) - x(v)\| = w(u, v).$$

If this function x exists, it is called a *realization* for G , we also refer to the realization as an *embedding* of G . For example, Figure 1(B) shows a realization with $K = 3$ for the graph in Figure 1(A), which does not have a realization when $K = 2$.

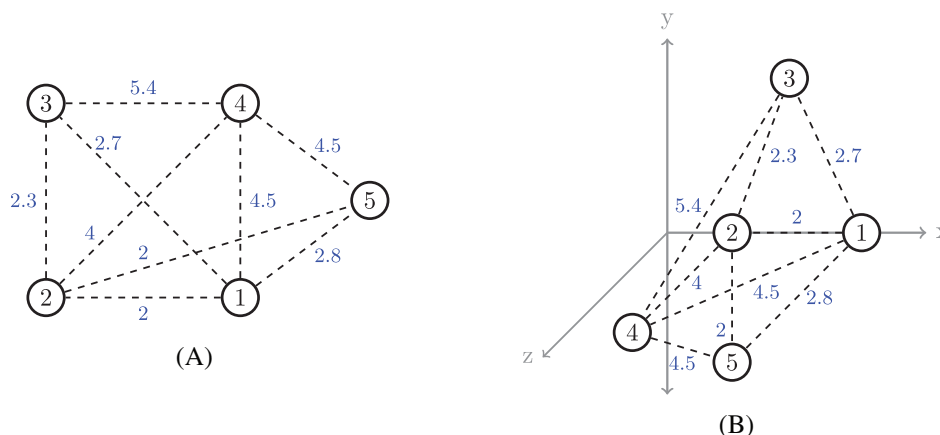


FIGURE 1 (A) A set of points, 1, 2, ..., 5, and known pairwise distances. (B) An embedding of these points in three dimensions

We assume G is connected, since determining if a disconnected graph has a valid realization is equivalent to determining if its connected components have a realization [1]. We also assume the norm in Definition 1 is the Euclidean norm for the remainder of this paper; however, this need not be the case. We also note that there exists a form of the problem where the function need not satisfy the strict equality in Definition 1, but rather must satisfy $w_l(u, v) \leq \|x(u) - x(v)\| \leq w_u(u, v)$ for lower and upper bounds on the weights $w_l(u, v)$ and $w_u(u, v)$, respectively; this is called the interval DGP [4, 10].

The DGP with integer weights is $\mathcal{NP}$ -Complete for $K = 1$ and $\mathcal{NP}$ -Hard for $K > 1$ [19], motivating the need for solution methods that are able to solve the problem in practice. Solution methods for the DGP include nonlinear programming, semi-definite programming, and the geometric build-up methods [6, 13, 15, 17]. If the distances between all pairs of vertices in G are known, that is, G is complete, and we assume a solution exists in $\mathbb{R}^K$, there is a procedure for finding the realization by solving a series of linear equations [18, Chapter 3].

In most applications, the input graph is not complete, that is, not all pairwise distances are known. In such a case, there exists an iterative procedure to position the vertices. In this procedure, we begin by choosing K vertices for which all pairwise distances are known; without loss of generality, we may position them at any coordinates which satisfy all their pairwise distances. Next, we choose a vertex with enough pairwise distances (at least K) relative to the previously positioned vertices and fix its position using this information. We repeat, fixing vertices with at least K pairwise distances to the previously fixed vertices, until we have a realization. This procedure is the idea behind the branch-and-prune (BP) algorithm, whose success depends on choosing the right fixing order [8, 13, 15, 16]. Thus we first aim to find the *order* in which the BP algorithm should iterate over vertices [9]. The class of DGP instances possessing such an order is called Discretizable DGPs (DDGPs). Otherwise, there may still be a realization, but the instance does not have enough distance information to determine one using the BP algorithm; as such it could be more difficult to find. We note that the input to these discretized ordering problems is an unweighted graph, and the output is a vertex order which is given as input together with the actual pairwise distances to the BP algorithm to find a realization. We direct the reader to [8, 13] for a formal definition and more detailed explanation of DDGPs and the BP algorithm.

In this paper, we study a key subclass of DDGPs, namely the Discretizable Molecular Distance Geometry Problem (DMDGP) [8] for which the literature is quite limited, as compared to other DDGPs. The ordering problem associated with DMDGP is the Contiguous Trilateration Ordering Problem (CTOP). Taking an unweighted graph as input, CTOP searches for an ordering of the vertices that ensures the first K vertices are pairwise adjacent and each subsequent vertex in the order is adjacent to the K *immediately* previously ranked vertices. The motivation for solving CTOP, instead of a relaxation of the problem, comes from the key application of realizing protein backbones with NMR data, since a certain structure is required to compute interatomic distances using covalent bond lengths and angles between them. These orders also have desirable properties: an order that is a solution to CTOP exhibits symmetries in the BP tree, so to find all solutions for a DMDGP the BP algorithm only needs to be applied to find a single solution and the rest can be enumerated using symmetry (Lavor et al. [8], Chapter 5). There are other ordering problems associated with DDGPs, one of the most commonly studied being the Discretizable Vertex Ordering Problem (DVOP) which relaxes the need for vertices to be adjacent to the immediately previous vertices to *any* vertex of lower rank. We remark that the solution to CTOP is a solution to DVOP but the converse is not true.

DVOP has been shown to be polynomially solvable via a greedy algorithm in fixed dimension [7]. Moreover, integer programming (IP) methods have been used to find DVOP orders which are optimal with respect to some measure of their BP tree size [18], and algorithms finding partial orders that optimize the BP search space have been proposed [2]. In contrast to DVOP, CTOP is shown to be $\mathcal{NP}$ -complete and several IP formulations have been proposed to solve it on small instances [1]. In [14], the same underlying graph structure as one of the IP formulations of Cassioli et al. [1] is used and an order is constructed using

a path in this graph. The algorithm is applied directly to small protein instances and the authors note it cannot be extended to large molecules. As such, we consider Cassioli et al. [1] the only work of comparison for our study.

The BP algorithm was first proposed for CTOP by Lavor et al. [9], and in [3] answer set programming was shown to improve the performance of the BP algorithm for CTOP. Furthermore, DMDGP vertex orders with repeated vertices, called re-orders, are considered [10], their computational complexity is analyzed [1] and further studied in detail [11].

The rest of the paper is organized as follows. In Section 2, we present in detail the CTOP and review two existing IP formulations from the literature. In Section 3, we introduce three novel constraint programming (CP) formulations for CTOP. We then present, in Section 4, a structural study of CTOP which may eventually aid in its solution. Finally, in Section 5, we present a computational study which compares the CP and IP models, and which demonstrates the current utility and drawbacks of the structural findings.

We note that an overview of our paper, namely the models from the literature as well as our proposed models and structural ideas, is provided in Table D1.

2 | PRELIMINARIES

In this section, we introduce the common notation used in the paper, provide the problem definition and briefly present the existing formulations for the problem.

2.1 | Notation

All sets are denoted calligraphically. Let $G = (\mathcal{V}, \mathcal{E})$ be an undirected graph, where $\mathcal{V}$ is the set of vertices and $\mathcal{E}$ is the set of edges. The *adjacency matrix* of G is denoted by A , i.e., $A_{v,u} = 1$ if and only if edge $\{u, v\} \in \mathcal{E}$. Denote the *neighborhood* of a vertex v as $\mathcal{N}(v)$, that is, $\mathcal{N}(v) = \{u \in \mathcal{V} : \{u, v\} \in \mathcal{E}\}$. Thus $v \notin \mathcal{N}(v)$ and the *degree* of v is $d(v) = |\mathcal{N}(v)|$. We let $G[\mathcal{V}'] = (\mathcal{V}', \mathcal{E}')$ be the subgraph of G induced by $\mathcal{V}' \subseteq \mathcal{V}$, and thus $\mathcal{E}' = \{\{u, v\} \in \mathcal{E} : u, v \in \mathcal{V}'\}$. A *clique*, $\mathcal{K}$, in G is a set of vertices $\{v_1, v_2, \dots, v_{|\mathcal{K}|}\} \subseteq \mathcal{V}$ such that $\{v_i, v_j\} \in \mathcal{E}$ for all $v_i, v_j \in \mathcal{K}$ with $v_i \neq v_j$. Similarly, a *stable set*, SS , in G is a set of vertices $\{u_1, \dots, u_{|SS|}\} \subseteq \mathcal{V}$ such that $\{u_i, u_j\} \notin \mathcal{E}$ for all $u_i, u_j \in SS$. We define an *adjacent predecessor* of a vertex $v \in \mathcal{V}$ as $u \in \mathcal{V}$ with $\{u, v\} \in \mathcal{E}$ such that u precedes v in a vertex order, and we define a *contiguous predecessor* of a vertex $v \in \mathcal{V}$ as $u \in \mathcal{V}$ with $\{u, v\} \in \mathcal{E}$, such that there is no $w \in \mathcal{V}$ with $\{w, v\} \in \mathcal{E}$ between v and u in the vertex order. Thus, u is an adjacent predecessor that immediately precedes v in the order, meaning there is no nonadjacent vertex between u and v in the vertex order.

For $a, b \in \mathbb{Z}_+$, $a \leq b$, we introduce the notation $[a] = \{0, 1, \dots, a\}$ and $[a, b] = \{a, a+1, \dots, b\}$. If $a > b$, then $[a, b] = \emptyset$, similarly if $a < 0$, then $[a] = \emptyset$. Indices follow these conventions: indices start at 0, so that the possible positions of a vertex order are $[|\mathcal{V}| - 1]$. We let $|\mathcal{V}| = n$, and use $|\mathcal{V}|$ in relation to vertices and n in relation to ranks, that is, positions, of a vertex order.

Finally, we introduce the set $\mathcal{V}^{d[K, K+\delta]} = \{v \in \mathcal{V} : d(v) \in [K, K+\delta]\}$ for some fixed positive integer δ , the set of vertices with degrees in $[K, K+\delta]$.

2.2 | Problem definition

Given an integer dimension K , the DMDGP [1] is the problem of finding a realization in $\mathbb{R}^K$ given a simple, connected, undirected graph instance, $G = (\mathcal{V}, \mathcal{E})$, possessing a total order of its vertices that satisfies the following:

- (i) the first K vertices in the order form a clique in the input graph G , and
- (ii) each vertex with rank $\geq K$ has at least K *contiguous predecessors*. That is, for each vertex at position $k \in [K, n-1]$ along with the vertices at positions $[k-K, k-1]$ form a $(K+1)$ -clique in the input graph.

We refer to a total order that satisfies (i) and (ii) as a *DMDGP order*, and a clique satisfying (i) as the *initial clique*. We say an instance for which a DMDGP order exists is *feasible*, otherwise it is *infeasible*. The problem of determining whether a DMDGP order exists for G is known as the Contiguous Trilateration Ordering Problem (CTOP) [1]. An instance of CTOP, i.e., an integer $K > 0$ and a simple, undirected, connected graph $G = (\mathcal{V}, \mathcal{E})$, will be denoted $(G = (\mathcal{V}, \mathcal{E}), K)$ or simply (G, K) . Cassioli et al. [1] proved CTOP is $\mathcal{NP}$ -complete.

We can restate points (i) and (ii) of the definition using a key property of the problem structure, as mentioned by Cassioli et al. [1]:

Definition 2 (DMDGP order). Given an integer dimension K , a DMDGP order is a total order of the vertices of a simple, connected, undirected graph $G = (\mathcal{V}, \mathcal{E})$, so that the order forms a series of $(K+1)$ -cliques which overlap by at least K vertices.

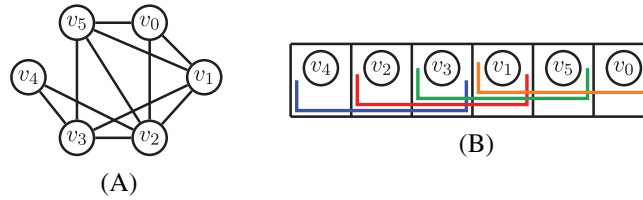


FIGURE 2 (A) A graph instance which is feasible for the contiguous trilateration ordering problem with $K = 2$. (B) Overlapping cliques of the order $(v_4, v_2, v_3, v_1, v_5, v_0)$

We note that there may be extra edges between vertices than those that form the overlapping cliques. This important property is depicted in Figure 2(B) for Example 1. Considering this overlapping clique structure, we refer to the first of those overlapping cliques as the *initial* $(K + 1)$ -clique.

Example 1. The graph given in Figure 2(A) with $K = 2$ is a feasible instance for CTOP. A possible DMDGP order is $(v_4, v_2, v_3, v_1, v_5, v_0)$. Clearly, since they are adjacent $\{v_4, v_2\}$ form a clique, v_3 is adjacent to both of its immediate predecessors: v_4 , and v_2 , so $\{v_4, v_2, v_3\}$ form a $(K + 1)$ -clique in the input graph. Similarly, v_1 is adjacent to v_2 , and v_3 , forming a $(K + 1)$ -clique in the input graph and so on.

2.3 | Existing IP models

Prior to this work, Cassioli et al. [1] present three IP formulations for CTOP. Below we summarize their properties, while we provide their full details for completeness in Appendix A.

- The vertex-rank formulation (IP^{VR}): They introduce $|\mathcal{V}| \times n$ binary variables indicating vertex-rank assignment. Then, the model contains $(|\mathcal{V}| + n)$ -many 1-1 assignment constraints and $(|\mathcal{V}| \times n)$ -many clique constraints.
- The clique digraph formulation (IP^{CD}): They enumerate all ordered cliques of size $(K + 1)$ in G , define a clique digraph D with vertices as those ordered cliques and arcs for pairs of cliques that suitably overlap to follow each other in the order (as in Figure 2(B)s). We note D is exactly the pseudo de Bruijn graph structure described in [14]. Then, the CTOP solution corresponds to a path in D . This IP model has digraph arc variables, first clique and last clique variables, and precedence variables¹ for vertices in G .
- The unordered clique relaxation (IP^{RELAX}): They relax the strict clique ordering constraints of the clique digraph formulation, and solve this relaxation as a first check for the existence of a DMDGP order. The benefit of this formulation is that it reduces the number of variables in (IP^{CD}), because we have reduced the worst-case number of vertices in the D by a factor of $(K + 1)!$. When a solution to (IP^{RELAX}) is found, it must be verified as this solution does not necessarily yield a DMDGP order. The verification is a simple check to ensure the linear order solution forms a DMDGP order. The strength in this formulation is that if (IP^{RELAX}) is infeasible, there is no DMDGP order for the instance.

Regarding the vertex-rank formulation (IP^{VR}), we prove that its LP relaxation is always feasible.

Proposition 1. *The LP relaxation of (IP^{VR}) on any instance $G = (\mathcal{V}, \mathcal{E})$ with $K \geq 2$ is feasible.*

Proof. See Appendix B.1. ■

This observation can be taken as a sign of the (IP^{VR}) model being weak. In fact, we observe in our computational experiments that especially for infeasible instances, a large number of branch-and-bound nodes are processed due to LP relaxations (and cuts derived from them) not being strong enough to prune infeasible branches early on.

On the other hand, we note that the clique digraph model (and its relaxation) mostly suffers from the large number of ordered (unordered) cliques, and hits either the time or memory limit in our numerical experiments.

Lastly, we note that as mentioned by Cassioli et al. [1], (IP^{VR}) works better for feasible instances, while (IP^{CD}) works better for infeasible instances. However, none of them scale well with the size of the input graph, which motivates our work on developing alternative formulations and study of the structure of DMDGP orders.

¹In [1], the variables for predecessors are given as w_{uv} , we believe this is a typo that the variables are in fact their y_{uv} variables which are our p_{uv} variables in the version provided in Appendix A.2.

3 | CP MODELS

CP is a natural approach to distance geometry ordering problems since CTOP is a Constraint Satisfiability Problem (CSP) where the optimal solution is one that merely satisfies all constraints [12]. Unlike Constraint Optimization Problems where the optimal solution minimizes or maximizes some objective function subject to the constraints, CSPs do not have an objective function. CP has been shown to work well for problems with a permutation structure [20] and allows the leveraging of global constraints such as AllDifferent. To our knowledge, no CP model for CTOP has ever been proposed. The flexibility of CP allows for three possible formulations for CTOP.

The first formulation follows naturally from $(\mathbb{IP}^{\text{VR}})$, from [1]. We define integer variables r_v equal to the rank of vertex $v \in \mathcal{V}$ in the order.

$$(\text{CP}^{\text{RANK}}) : \text{AllDifferent}(r_0, r_1, \dots, r_{|\mathcal{V}|-1}) \quad (1a)$$

$$|r_u - r_v| \geq K + 1 \quad \forall u, v \in \mathcal{V} \text{ s.t. } u \neq v, \{u, v\} \notin \mathcal{E} \quad (1b)$$

$$r_v \in [n - 1] \quad \forall v \in \mathcal{V} \quad (1c)$$

Using the global constraint AllDifferent (1a), we enforce that each vertex has a unique rank. Together with the domain constraints, (1c), this is equivalent to the one-to-one assignment constraints, in $(\mathbb{IP}^{\text{VR}})$, since each rank has a possible domain of $[n - 1]$ and we are enforcing the constraint over all the rank variables which are indexed by the vertices, that is, $|\mathcal{V}| = n$ variables. To enforce clique constraints, (1b), we use the idea that if two vertices do not have an edge between them, they cannot be in the same $(K + 1)$ -clique. In other words their ranks must have a difference of at least $K + 1$. This constraint completely models the clique constraints and the predecessor constraints since if their rank difference is $\leq K$ then vertices u and v must be in the same clique which contradicts there being no edge between them. We note constraints (1b) can also be expressed using a minimum distance constraint; however, our experiments show that the constraints as stated above had better performance.

Secondly, we present what is called a dual formulation in CP [20], where the values and variable meanings are swapped. Let integer variable v_r represent the vertex in position r of the order and let $\text{table}(\mathcal{E})$ denote the table which contains a tuple for each edge in $\mathcal{E}$.

$$(\text{CP}^{\text{VERTEX}}) : \text{AllDifferent}(v_0, v_1, \dots, v_{n-1}) \quad (2a)$$

$$(v_i, v_j) \in \text{table}(\mathcal{E}) \quad \forall i \in [0, n - K - 1], j \in [i + 1, i + K] \quad (2b)$$

$$v_r \in [|\mathcal{V}| - 1] \quad \forall r \in [n - 1] \quad (2c)$$

In (2a) we enforce that each rank has a unique vertex, again using AllDifferent. To enforce the clique and predecessor constraints (2b), we use the CP notion of *table constraints*. These are global constraints that can be seen as an extension of domain constraints for a set of decision variables, where only certain predefined value combinations are allowed for those variables. Explicitly capturing some relationships between variables, they lead to improved propagation in the CP solver. In constraints (2b), in order to ensure the overlapping clique structure required, we enforce that for any pair of ranks that differ by at most K , the vertices assigned to these positions in the order must be adjacent in G , thus they should correspond to the two endpoints of an edge in G . Finally, (2c) enforces the domain of the variables.

The last CP model is the result of combining the rank and vertex models into a single model by *channelling* the variables using an *inverse constraint*. It uses the constraints for predecessors and cliques from both formulations. This is useful because redundant constraints may actually help CP solvers perform more inference and discover feasible solutions in a shorter amount of time. Having defined v and r variables as before, the combined model is as follows:

$$(\text{CP}^{\text{COMBINED}}) : \text{AllDifferent}(v_0, v_1, \dots, v_{n-1}) \quad (3a)$$

$$\text{AllDifferent}(r_0, r_1, \dots, r_{|\mathcal{V}|-1}) \quad (3b)$$

$$|r_u - r_v| \geq K + 1 \quad \forall u, v \in \mathcal{V} \text{ s.t. } u \neq v, \{u, v\} \notin \mathcal{E} \quad (3c)$$

$$(v_i, v_j) \in \text{table}(\mathcal{E}) \quad \forall i \in [0, n - K - 1], j \in [i + 1, i + K] \quad (3d)$$

$$\text{inverse}(r, v) \quad (3e)$$

$$r_v \in [n - 1] \quad \forall v \in \mathcal{V} \quad (3f)$$

$$v_r \in [|\mathcal{V}| - 1] \quad \forall r \in [n - 1] \quad (3g)$$

In this formulation, the inverse constraint (3e) enforces the relation $(r_u = j) \equiv (v_j = u)$, which also makes the AllDifferent constraints in the vertex and rank models redundant. However we include them in the model, as when using the appropriate CP parameters they improve the results.

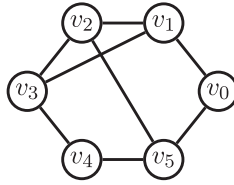


FIGURE 3 A graph which is infeasible for Contiguous Trilateration Ordering Problem with $K = 2$ and $K = 3$

4 | STRUCTURAL ANALYSIS OF CTOP

In this section we present a study of the structure of DMDGP orders that we believe can lead to improvements for the formulations presented in Section 3. We present these structural findings as checks for infeasible instances, procedures for reducing the domains and breaking symmetries in DMDGP orders, and a class of valid inequalities.

4.1 | Infeasibility checks

We begin the discussion of the structure of CTOP by introducing some simple checks which will immediately indicate if an instance (G, K) is infeasible. The first check arises from the fact that every vertex needs at least K neighbors to be a part of a $(K + 1)$ -clique.

Infeasibility Check 1 (Minimum degree). *Given (G, K) , if $\exists v \in \mathcal{V}$ such that $d(v) < K$ then G does not have a DMDGP order for K .*

Similarly, it is possible to determine a lower bound on the number of edges in G .

Proposition 2. *Given (G, K) , the minimum number of edges in G to have a DMDGP order is $\left(|\mathcal{V}| - \frac{1}{2}\right)K - \frac{1}{2}K^2$.*

Proof. See Appendix B.2. ■

Proposition 2 leads to a second check for infeasibility.

Infeasibility Check 2 (Minimum edges). *Given (G, K) , if $|\mathcal{E}| < \left(|\mathcal{V}| - \frac{1}{2}\right)K - \frac{1}{2}K^2$ then this instance is infeasible.*

Example 2. The graph in Figure 3 is infeasible with $K = 2$ and $K = 3$. For $K = 2$, the instance passes Infeasibility Check 1 as every vertex has at least two neighbors. However Infeasibility Check 2 proves it is infeasible as the graph has $|\mathcal{E}| = 8$ and the minimum number of edges for $K = 2$ is $(6 - 0.5)2 - 0.5(2^2) = 9$. For $K = 3$, we can prove this instance is infeasible using Infeasibility Check 1 since $d(v_4) = 2$.

Cassioli et al. [1] establish a lower bound on the degree of a vertex, which depends on its position in the order. If a vertex has degree K then it can only be placed in the first or last position, since it can only appear in a single $(K + 1)$ -clique. If there are more than two vertices with degree exactly K the instance must be infeasible as there are only two available positions for these vertices. Similarly, there are four positions available for a vertex with degree $K + 1$; ranks 0, 1, $n - 2$, $n - 1$, so if there are more than four vertices with degree $K + 1$, the instance is infeasible. This argument can be extended to the frequency of all vertices of degree strictly less than $2K$. We formalize the argument of Cassioli et al. [1] as Infeasibility Checks. We introduce the set $\mathcal{V}^{d[K, K+\delta]} = \{v \in \mathcal{V} \mid d(v) \in [K, K + \delta]\}$ for some fixed positive integer δ , to help express these arguments. We call vertices v with $d(v) < 2K$ *small degree vertices* and vertices v with $d(v) \geq 2K$ *large degree vertices*.

Infeasibility Check 3 (Upper bound on small degree vertices). *Given an instance (G, K) , if $\exists \delta \in [K - 1]$ such that $|\mathcal{V}^{d[K, K+\delta]}| \geq 2(\delta + 1) + 1$ then this instance is infeasible.*

Example 3. Consider the graph in Figure 4(A) with $K = 3$. We will see that it is infeasible by Infeasibility Check 3. Note that this instance cannot be proved infeasible by Infeasibility Check 1 but can be proved infeasible by Infeasibility Check 2. Since $K = 3$, we have $\delta \in [1, 2]$. First let $\delta = 1$, we have

$$\mathcal{V}^{d[3,4]} = \{v_0, v_1, v_2, v_3, v_4\},$$

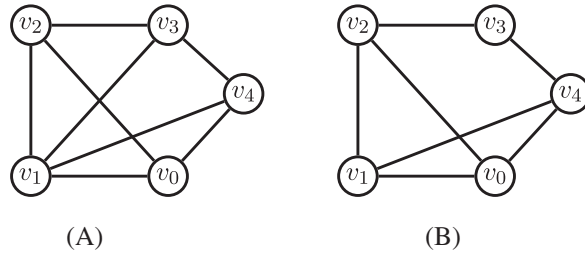


FIGURE 4 Two graphs that are infeasible for the contiguous trilateration ordering problem. (A) Infeasible with $K = 3$; (B) Infeasible with $K = 2$

and $|\mathcal{V}^{d[3,4]}| = 5$. The right-hand side of the expression in Infeasibility Check 3 with $\delta = 1$ is

$$2(1 + 1) + 1 = 5,$$

and so we are able to say this instance is infeasible.

Similarly, we have a lower bound on the number of vertices with larger degree. Since the central $(n - 2K)$ vertices in the order are in at least $2K$ cliques, they must all have degree of at least $2K$, meaning there must be enough vertices with large degree to occupy these $(n - 2K)$ positions.

Infeasibility Check 4 (Lower bound on large degree vertices). *Given (G, K) with $n \geq (2K + 1)$, if $|\mathcal{V}^{d[2K, n-1]}| \leq n - (2K + 1)$ then the instance is infeasible.*

Example 4. Consider the graph in Figure 4(B) with $K = 2$. We will see that it is infeasible by Infeasibility Check 4. Note that this instance cannot be proven infeasible by Infeasibility Check 1 or by Infeasibility Check 2. We have $n = 5 = 2K + 1$ and so $\mathcal{V}^{d[4,4]} = \emptyset$ thus $|\mathcal{V}^{d[4,4]}| = 0$. We also have $n - 2K - 1 = 0$, and so we are able to say this instance is infeasible.

4.2 | Domain reduction

We are able to exploit some structural characteristics of CTOP to help prune variable domains in the CP formulations. Let the domain of an integer variable x be given by D_x .

First, we extend the lower bounds on the degree of a vertex given by Cassioli et al. [1] to set the domains for rank variables. As observed previously, a DMDGP order is a series of overlapping cliques of size (at least) $K + 1$. In the minimal case, the first and last vertices in the order are in exactly one clique, the second and second to last vertices are in two cliques, and so on. The central $(|\mathcal{V}| - 2K)$ vertices are in at least $2K$ cliques. From this we can infer the minimum number of neighbours required by a vertex at a given rank.

Domain Reduction Rule 1 (Domain reduction for small degree vertices). *Given an instance (G, K) , we can define the domain for the rank variables as follows:*

$$D_{r_v} = \begin{cases} [d(v) - K] \cup [n - 1 - (d(v) - K), n - 1] & \text{if } d(v) < 2K \\ [n - 1] & \text{otherwise} \end{cases}$$

Example 5. Consider the graph in Figure 5 with $K = 2$. It has two vertices with degree strictly less than $2K$: v_0 and v_4 . By Domain Reduction Rule 1 we can reduce the domain of both vertices so that their new domains are $D_{r_{v_0}} = \{0, 1, 4, 5\}$ and $D_{r_{v_4}} = \{0, 5\}$.

We are able to extend domain reduction to the vertices that are adjacent to small degree vertices. The intuition is that if a vertex has small degree, the position of its neighbors cannot be too far from that vertex. If the position of a small degree vertex v^* has already been limited, its neighbors must be within the first or the last $d(v^*)$ vertices of the order since they are all connected to v^* .

Domain Reduction Rule 2 (Domain reduction for neighborhood of small degree vertices). *Given an instance (G, K) with $n \geq (2K + 1)$, for all $v^* \in \mathcal{V}^{d[K, 2K-1]}$*

$$D_{r_v} = [d(v^*)] \cup [n - 1 - d(v^*), n - 1] \quad \forall v \in \mathcal{N}(v^*).$$

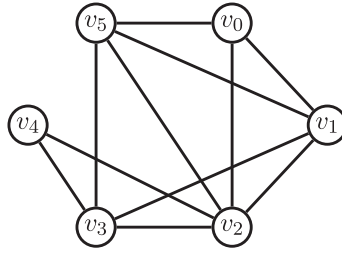


FIGURE 5 A graph instance feasible for contiguous trilateration ordering problem with $K = 2$, to illustrate Domain Reduction Rule 1

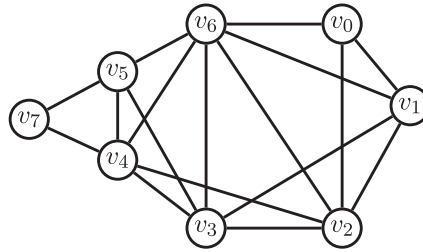


FIGURE 6 A graph instance feasible for contiguous trilateration ordering problem with $K = 2$, to illustrate Domain Reduction Rule 2

Example 6. Consider the graph in Figure 6 with $K = 2$. For Domain Reduction Rule 2, we have $\mathcal{V}^{d[2,3]} = \{v_0, v_7\}$. The neighbours of v_7 are v_5 and v_4 . Since they both have degree greater than $2K$, their domains would not have been reduced by Domain Reduction 1. We reduce their domains as follows

$$D_{r_{v_5}} = D_{r_{v_4}} = \{0, 1, 2, 5, 6, 7\}.$$

For the neighbors of v_0 , we notice that $[d(v^*)] \cup [n - 1 - d(v^*), n - 1] = [n - 1]$, so we will not be able to reduce their domains.

4.3 | Symmetry breaking

As observed by Cassioli et al. [1], reversing a DMDGP order also gives a DMDGP order. We establish that these are not the only symmetries present in DMDGP orders, and present strategies for breaking these symmetries. We begin by a simple condition to break the reverse symmetry. First, notice that if there is a single vertex that has degree K without loss of generality we can fix its position to 0. If there is a second vertex with degree K we can fix its position to $n - 1$, noting that there are at most two vertices of degree K in a DMDGP order due to Infeasibility Check 3.

Symmetry Breaking Condition 1 (Degree K). If $\mathcal{V}^{d[K,K]} = \{v_i\}$, then let $r_{v_i} = 0$. If $\mathcal{V}^{d[K,K]} = \{v_i, v_j\}$, then let $r_{v_i} = 0$ and $r_{v_j} = n - 1$.

Next, we observe that if two vertices have the same neighbourhood excluding each other, they are interchangeable in the DMDGP order since they will have exactly the same contiguous predecessors. This guarantees a DMDGP order, since the only condition we need to meet to preserve the order if we interchange two vertices is ensuring that they have the appropriate contiguous predecessors. We call this symmetry *pairwise symmetry*, which can be broken by imposing an arbitrary order on the pair of such symmetric vertices. Ideally, we would identify a large set of such vertices and order them. However, identifying such vertex sets can be computationally expensive. We instead identify two types of vertex sets that will allow for easy detection and breaking of pairwise symmetry. Specifically, we consider stable sets and cliques in the input graph.

Symmetry Breaking Condition 2 (Stable set). For a stable set $SS = \{v_1, v_2, \dots, v_k\} \subseteq \mathcal{V}$ such that $\mathcal{N}(v_i) = \mathcal{N}(v_j) \forall v_i, v_j \in SS$ we enforce that $r_{v_1} < r_{v_2} < \dots < r_{v_k}$.

Symmetry Breaking Condition 3 (Clique). For a clique $\mathcal{K} = \{v_1, v_2, \dots, v_k\} \subseteq \mathcal{V}$ such that $\mathcal{N}(v_i) \setminus \mathcal{K} = \mathcal{N}(v_j) \setminus \mathcal{K}, \forall v_i, v_j \in \mathcal{K}$ we enforce that $r_{v_1} < r_{v_2} < \dots < r_{v_k}$.

In our experiments, we examine only cliques of size three or less, since we are usually unable to find large cliques satisfying Condition 3. Furthermore, we are able to conditionally extend these symmetry breaking conditions to include more vertices. Consider, for example, two vertices v and u whose neighborhoods differ only by one vertex $w \in \mathcal{N}(v)$. If in the DMDGP order w is at least $K + 1$ positions away from v , the edge connecting them is not necessary to enforce precedence in the order, that is, w is not an contiguous predecessor of v and vice versa. In this case we can essentially consider u and v as having the same neighborhood and so can impose symmetry breaking on them. For some set $S \subseteq \mathcal{V}$ we denote $\mathcal{N}(S) = \cup_{v \in S} \mathcal{N}(v) \setminus S$, the set of all vertices, outside of S that are adjacent to a vertex in S .

Symmetry Breaking Condition 4 (Extended stable set). *Let SS be a stable set meeting Condition 2 or a single vertex not in any stable set meeting Condition 2. For a vertex $v \in \mathcal{V} \setminus (SS \cup \mathcal{N}(SS))$ such that $\mathcal{N}(v) \setminus \mathcal{N}(SS) = \{w\}$ we enforce the logical constraints:*

$$|r_v - r_w| \geq K + 1 \Rightarrow r_v < r_u \quad \forall u \in SS.$$

If we have enforced an ordering for SS already, we need only add the constraint

$$|r_v - r_w| \geq K + 1 \Rightarrow r_v < r_{v_1}.$$

Symmetry Breaking Condition 5 (Extended clique). *Let $\mathcal{K}$ be a clique meeting Condition 3 or a single vertex not in any clique meeting Condition 3. For a vertex $v \in \mathcal{N}(\mathcal{K})$ such that $(\mathcal{N}(v) \cup \{v\}) \setminus (\mathcal{N}(\mathcal{K}) \cup \mathcal{K}) = \{w\}$ we enforce the logical constraints*

$$|r_v - r_w| \geq K + 1 \Rightarrow r_v < r_u \quad \forall u \in \mathcal{K}.$$

Finally, if we have not been able to break any symmetry via any of the previous ways we can arbitrarily choose two vertices and impose an order on them.

Symmetry Breaking Condition 6 (Arbitrary). *For any $v_1, v_2 \in \mathcal{V}$ enforce that $r_{v_1} < r_{v_2}$.*

An example of the strength of symmetry breaking with $K = 2$ can be found in Appendix C.

In our experiments, we first test for Symmetry Breaking Conditions 2 and 3 and then for Symmetry Breaking Conditions 4 and 5, breaking the symmetry where possible. If none of these conditions are met, we test for symmetry using Symmetry Breaking Condition 1, because it is unlikely we will have a vertex with degree exactly K if n is large. Finally, if all previous Symmetry Breaking Conditions have failed, we break the symmetry using Symmetry Breaking Condition 6.

4.4 | A class of valid inequalities

Next, we develop the structure of a class of valid inequalities for CTOP. We note that the focus of this study is not the practical application of these valid inequalities but the underlying structural analysis that leads to them. We proceed with the following intuition: if we identify some subset $S \subseteq \mathcal{V}$ such that the induced subgraph of S does not have a DMDGP order, the entire set S cannot appear consecutively in the order.

If for a given instance, (G, K) , we are able to identify subsets $S \subseteq \mathcal{V}$ whose induced graphs, $G[S]$, do not have DMDGP orders for K , we can add cuts to enforce that the difference between the maximum rank and the minimum rank of any element in S is at least $|S|$. Let $r_{\max}, r_{\min}$ denote the maximum rank and the minimum rank of any vertex in S , respectively. The valid inequality is

$$r_{\max} - r_{\min} \geq |S|. \quad (4)$$

We can improve this cut by examining the vertex in S with the smallest degree in the induced subgraph. Let $\mathcal{E}[S]$ be the edge set of $G[S]$, and let $\delta_S^{\text{miss}}(v) = |\{u \in S \setminus \{v\} \mid (v, u) \notin \mathcal{E}[S]\}|$, that is, the number of edges with one endpoint at $v \in S$ missing from $G[S]$ and let $\delta_S^{\text{miss}} = \max_{v \in S} \delta_S^{\text{miss}}(v)$. If $K > |S| - \delta_S^{\text{miss}}$, the difference between the maximum rank and the minimum rank must be greater than $\delta_S^{\text{miss}} + K$, because the $v \in S$ which has δ_S^{miss} , cannot be in a clique with δ_S^{miss} of the vertices in S , so we need at minimum δ_S^{miss} extra vertices between the vertices of S in the order. Otherwise, if $K \leq |S| - \delta_S^{\text{miss}}$, the difference in ranks must be greater than $|S|$ which is the inequality (4). So, the valid inequality is

$$r_{\max} - r_{\min} \geq \max\{|S|, \delta_S^{\text{miss}} + K\}. \quad (5)$$

Example 7. For the graph in Figure 7 with $K = 3$, consider $S = \{v_0, v_3, v_4, v_5\}$ where $G[S]$ has no DMDGP order since $|\mathcal{E}[S]| = 4$ and Infeasibility Check 2 gives a lower bound of six edges for a DMDGP order. We have

$$\delta_S^{\text{miss}} = \max\{\delta_S^{\text{miss}}(v_0), \delta_S^{\text{miss}}(v_3), \delta_S^{\text{miss}}(v_4), \delta_S^{\text{miss}}(v_5)\}$$

$$\delta_S^{\text{miss}} = \max\{2, 0, 1, 1\}$$

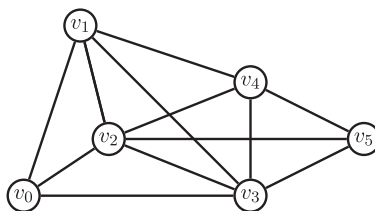


FIGURE 7 Graph with discretizable molecular distance geometry problem order $(v_5, v_4, v_3, v_2, v_1, v_0)$ for $K = 3$

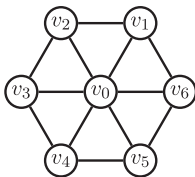


FIGURE 8 A wheel graph with seven vertices, W_7

so we have $\max\{|S|, \delta_S^{miss} + K\} = \max\{4, 2 + 3\} = 5$. Thus (5) gives:

$$r_{\max} - r_{\min} \geq 5$$

which is stronger than the original (4),

$$r_{\max} - r_{\min} \geq 4.$$

Note that this may not have been the case with a different choice of S .

The task of finding subsets of vertices S so that the subgraph induced by S does not have a DMDGP order is as difficult as determining if the whole graph has a DMDGP order. Thus, we would like to find sets of vertices with the most edges missing in their induced subgraph. As the sets with the most missing edges are stable sets, we can consider stable sets in G as candidate S sets. For any stable set SS , no pair of vertices can appear in the same $(K + 1)$ -clique. Thus, each pair of vertices in SS needs to have a difference in their ranks of at least $K + 1$, meaning the minimum rank and maximum rank must have a difference of $(|SS| - 1)(K + 1)$. The inequality becomes

$$r_{\max} - r_{\min} \geq (|SS| - 1)(K + 1). \quad (6)$$

Example 8. Using the graph from Figure 7 and $K = 3$, consider $SS = \{v_0, v_5\}$. The inequality (6) is:

$$r_{\max} - r_{\min} \geq (2 - 1)(3 + 1)$$

$$r_{\max} - r_{\min} \geq 4.$$

This observation also yields a simple check for infeasibility.

Infeasibility Check 5 Given (G, K) , if the size of the maximum stable set in G is greater than $\frac{n}{K+1} + 1$, we can immediately say G does not have a DMDGP order with K .

Proposition 3. The inequalities (5) and (6) are incomparable.

Proof. See Appendix B.3. ■

Example 9. Take the wheel graph W_7 , as pictured in Figure 8, with $K = 3$. The maximum stable sets are $\{1, 3, 5\}$ and $\{2, 4, 6\}$ which have size 3. The right-hand side of the inequality in Infeasibility Check 5 is $\frac{7}{3+1} + 1 = 2.75 \leq 3$. Thus we can say immediately that this instance is infeasible.

In fact, we are able to generalize this for all wheel graphs.

TABLE 1 Random graph instances

	n	D	Number of instances	Number of feasible	Number of infeasible	Number of unsolved
Small	20–60	0.3	27	0	27	0
		0.5	27	20	7	0
		0.7	27	27	0	0
Total			81	47	34	0
Medium	65–100	0.2	24	0	24	0
		0.3	24	0	10	14
		0.4	24	13	0	11
		0.5	24	24	0	0
Total			96	37	34	25

Proposition 4. For any wheel graph, W_n , with $K \geq 2$, if n is odd and $n \geq \frac{K+1}{K-1}$ or if n is even and $n \geq 2\frac{K+1}{K-1}$ there is no DMDGP order.

Proof. See Appendix B3. ■

Finally, we define these valid inequalities so that they may be added to the $(\text{CP}^{\text{RANK}})$ and $(\text{CP}^{\text{COMBINED}})$ formulations. Given a stable set $SS \subseteq \mathcal{V}$, and the rank variables r_v we have two options to implement the valid inequalities, as follows:

$$\max\{r_v | v \in SS\} - \min\{r_v | v \in SS\} \geq (|SS| - 1)(K + 1) \quad (7)$$

$$\text{MinDistance}(\{r_v | v \in SS\}, K + 1) \quad (8)$$

where the constraint (8) is a minimum distance constraint which ensures that all variables it acts upon take values at least $K + 1$ apart from each other.

5 | COMPUTATIONAL RESULTS

5.1 | Experimental setup

To solve the IPs we use the solver IBM ILOG CPLEX version 12.8.0 and to solve the CPs we use IBM ILOG CP Optimizer version 12.8.0. All models were implemented in C++ and run on MacOs with 16 GB RAM and a 2.3 GHz Intel Core i5 processor, using a single thread. The default IP and CP parameters were used with the exception of in $(\text{CP}^{\text{COMBINED}})$ where extended inference was invoked on the AllDifferent constraints. We use $K = 3$ for all experiments as this is the value of K frequently used in applications. The time limit is set to 7200 s. (We also tested the small random instances with $K = 4$ and $K = 5$; their results are provided in Tables 10 and 11 of Appendix S1, respectively. We note the results were similar to those with $K = 3$.)

5.2 | Instances

We perform our numerical experiments on a test dataset consisting of randomly generated graphs² and pseudo-protein graphs³.

5.2.1 | Random instances

We divide the random test set into *small instances*, having $n \in \{20, 25, \dots, 60\}$ and the expected edge density (measured as $D = \frac{2|E|}{n(n-1)}$) in $\{0.3, 0.5, 0.7\}$, and *medium-sized instances* which have $n \in \{65, 70, \dots, 100\}$ and the expected edge density in $\{0.2, 0.3, 0.4, 0.5\}$. For each n , density pair, we generate three graph instances using the `dense_gnm_random_graph()` method in the NetworkX Python package [5], which chooses a graph uniformly at random from the set of all graphs with n vertices and m edges. These are Erdős–Rényi random $G(n, M)$ graphs. Table 1 presents a summary of the instances. We remark that a portion of the instances were unsolved by any method; we denote these as *unsolved* instances.

We remark that for the small instance dataset, all 27 graph instances with $D = 0.3$ were infeasible and all 27 graph instances with $D = 0.7$ were feasible. When $D = 0.5$, we have 7 instances which are infeasible, all of which have $n = 20, n = 25$

²Random instances can be found at https://sites.google.com/site/mervebodur/home/DMDGP_Instances.zip.

³Pseudo-protein instances can be found at <https://gitlab.insa-rennes.fr/Jeremy.Omer/MinDouble>.

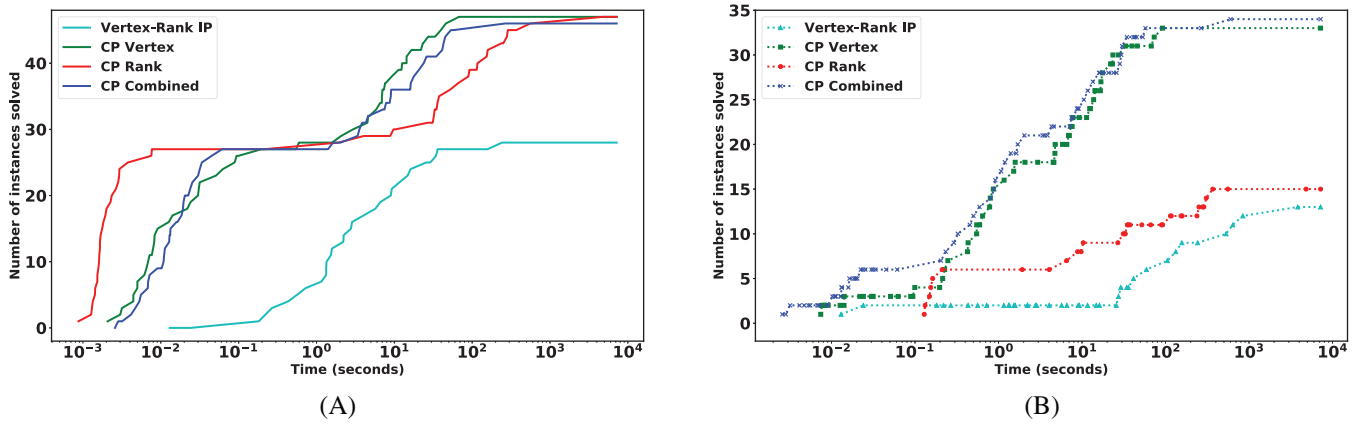


FIGURE 9 Solution times of the models for small instances. (A) Small feasible instances; (B) Small infeasible instances

or $n = 30$, the 20 feasible instances have $n \geq 30$. For medium-sized instances, preliminary results showed all instances with $D > 0.5$ were feasible and solved in less than a second. For this reason we focus our study on instances with $D \leq 0.5$, which give more insights into the solution methods. The 24 instances with $D = 0.2$ were infeasible and the 24 instances with $D = 0.5$ were feasible. The instances with $D = 0.3$ and $D = 0.4$ were more difficult to solve. For instances with $D = 0.3$, we found 10 infeasible instances, those with $n = 65, 70, 75, 80$, with all others unsolved. For medium-sized instances with $D = 0.4$, 13 instances were feasible with 11 others unsolved.

5.2.2 | Pseudo-protein instances

The pseudo-protein instances are similar to those used in [18] and were provided by the authors of that paper. They are generated using existing instances of the Protein Data Bank, which are cut to the required number of vertices and then reduced in density by randomly removing edges. This test dataset has 209 instances with vertices in $\{30, 40, 50, 60\}$ and expected edge density in $[0.03, 0.12]$. We remark that of these pseudo-protein instances 192 are infeasible and 17 are feasible.

5.3 | Computational results and discussion

In this section, we provide our observations based on a thorough computational study. All the detailed experimental result tables are provided in Appendix S1. Here, we summarize our main findings by first outlining the results, without the addition of improvements from Section 4, of the novel CP formulations as compared to the IP formulations from the literature. Then in Sections 5.4 and 5.5 we present a preliminary study of the structural findings and valid inequalities, respectively.

We begin by noting that the IP formulations from the literature do not perform well against the CP formulations, which can be seen in detail in Table 1. (IP^{CD}) is able to solve instances with $D = 0.3$ and $n \leq 40$ in less than a second. These are also the instances that are infeasible. However, for higher densities, and as n increases, (IP^{CD}) either hits the time limit or memory limit, with 50% of the instances hitting the time or memory limit for (IP^{CD}) , due to the large number of ordered cliques for larger and more dense instances. On average the instances have 165 875 ordered cliques, with the smallest number of ordered cliques being 48 and the largest being 1 406 256 cliques. Table S3 shows that $(\text{IP}^{\text{RELAX}})$ has performance similar to (IP^{CD}) , performing best when the number of vertices is small and when the density is low. For $n \geq 30$, $(\text{IP}^{\text{RELAX}})$ is unable to solve any feasible instance (all of which have $D = 0.5$ or $D = 0.7$) without running out of time or memory. In fact, when the time limit is hit, we are still in the root node of the tree, for instances with $n \geq 50$. Even when we relax the ordering constraints on the cliques, we still have a large number of unordered cliques. The smallest number of unordered cliques for an instance is 2 and the largest is 58 594, on average an instance has 6911 unordered cliques. (IP^{VR}) begins to hit the time limit at $n = 25$ and for $n \geq 35$ it is only able to solve one instance with $D < 0.7$. This confirms the observations of [1], but we have also shown that neither (IP^{CD}) nor (IP^{VR}) scale well.

We next compare our CP formulations with the vertex-rank IP formulation (IP^{VR}) of Cassioli et al. [1]. In Figure 9, we provide performance profiles for the solutions times of different models on small instances. Note that solution times are given in a logarithmic scale. We observe that the CP formulations all outperform (IP^{VR}) . For small feasible instances, as seen in Figure 9(A), $(\text{CP}^{\text{RANK}})$ is able to solve 29 instances in less than a second. However, for feasible instances $(\text{CP}^{\text{COMBINED}})$ and $(\text{CP}^{\text{VERTEX}})$ perform the best overall, with $(\text{CP}^{\text{VERTEX}})$ solving one more instances than $(\text{CP}^{\text{COMBINED}})$ within the time limit. The performance profile for small infeasible instances in Figure 9(B) shows that $(\text{CP}^{\text{COMBINED}})$ has the best performance on these instances.

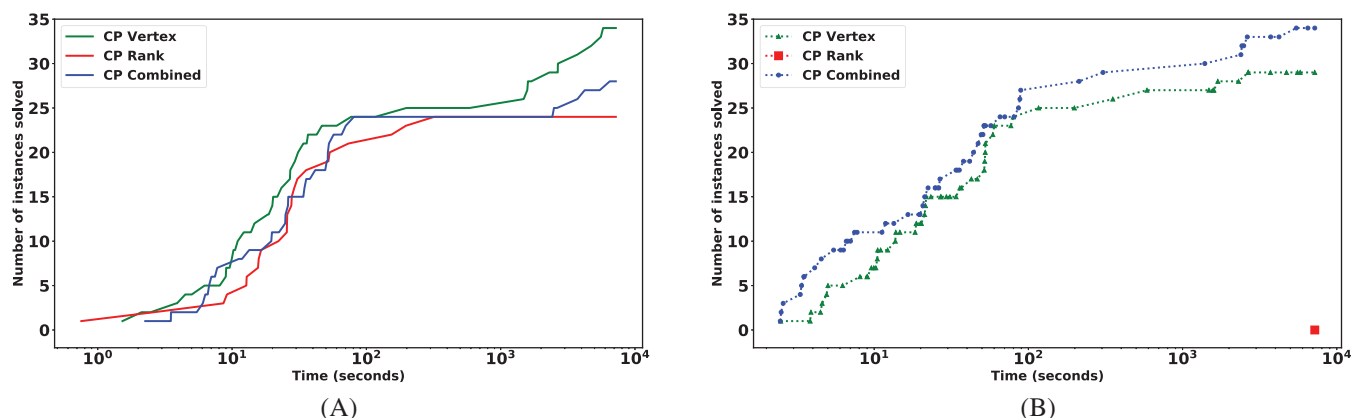


FIGURE 10 Solution times of the constraint programming models for medium-sized instances. (A) Medium-sized feasible instances; (B) Medium-sized infeasible instances

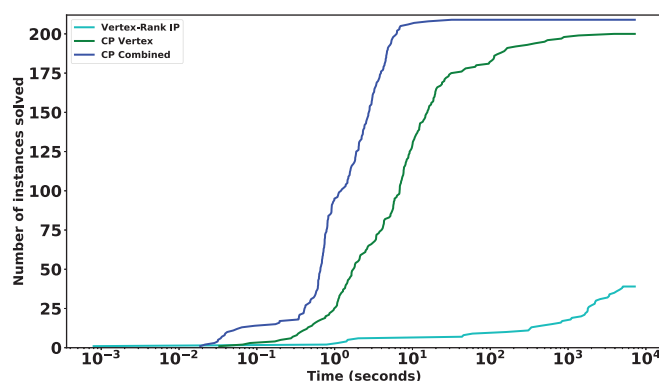


FIGURE 11 Solution times for pseudo-protein instances

Table S1 also reveals that (CP^{RANK}) is able to solve instances with $D = 0.7$ in less than a second, however, it begins to hit the time limit for $n \geq 35$ when $D = 0.3$. For $D = 0.5$, (CP^{RANK}) is able to solve instances but is almost always outperformed by ($\text{CP}^{\text{COMBINED}}$) and ($\text{CP}^{\text{VERTEX}}$). We also remark that after 25 vertices, with $D = 0.5$, the number of choice points for solving with (CP^{RANK}) exceed one million for all but two instances.

Overall, we conclude lower density instances are more challenging for the CP models. However, high density ($D = 0.7$) is trivial even with 60 vertices. For these reasons, we focus on densities less than or equal to 0.5, but increase the number of densities considered. Due to their poor performance, we exclude the IPs from further study. We now direct our focus to the CP models for medium-sized instances.

The performance profiles for the solutions times of different models on medium-sized instances are given in Figure 10 and the detailed results table can be found in Table S2. We observe that (CP^{RANK}) is outperformed by ($\text{CP}^{\text{COMBINED}}$) and ($\text{CP}^{\text{VERTEX}}$) in both the feasible and infeasible cases; it is unable to solve any infeasible instances. In the feasible case, ($\text{CP}^{\text{VERTEX}}$) solves 6 more instances than ($\text{CP}^{\text{COMBINED}}$) within the time limit, however in the infeasible case, ($\text{CP}^{\text{COMBINED}}$) solves 5 more instances than ($\text{CP}^{\text{VERTEX}}$) within the time limit. Overall, ($\text{CP}^{\text{VERTEX}}$) is able to solve one more instance than ($\text{CP}^{\text{COMBINED}}$) before the time limit is reached. We note however, that for medium-sized instances 25 of the instances were unsolved by any method. For these medium-sized instances, those with $D = 0.2$ or $D = 0.5$ were all solved in less than 2 min. The $D = 0.3$ and $D = 0.4$ instances are more difficult with only 13 of 24 instances with $D = 0.4$ solved and no instances with $D = 0.3$ and $n \geq 85$ solved. Thus there is still opportunity to improve the CP formulations.

Finally, we compare the best-performing CP formulations, ($\text{CP}^{\text{COMBINED}}$) and ($\text{CP}^{\text{VERTEX}}$), with (IP^{VR}) on the pseudo-protein instances. As seen in Figure 11, the results can be found in Tables S4–S7. We observe that ($\text{CP}^{\text{VERTEX}}$) and ($\text{CP}^{\text{COMBINED}}$) outperform (IP^{VR}), which is only able to solve 39 instances in total within the time limit. ($\text{CP}^{\text{COMBINED}}$) has the best performance on these instances solving every instance in less than 32 s. ($\text{CP}^{\text{VERTEX}}$) is unable to solve 9 instances before the time limit is hit. We observe that of the instances ($\text{CP}^{\text{VERTEX}}$) is unable to solve, eight have $n \geq 40$ and they are those that have the highest density. Overall, we conclude that the CP formulations perform very well on pseudo-protein instances.

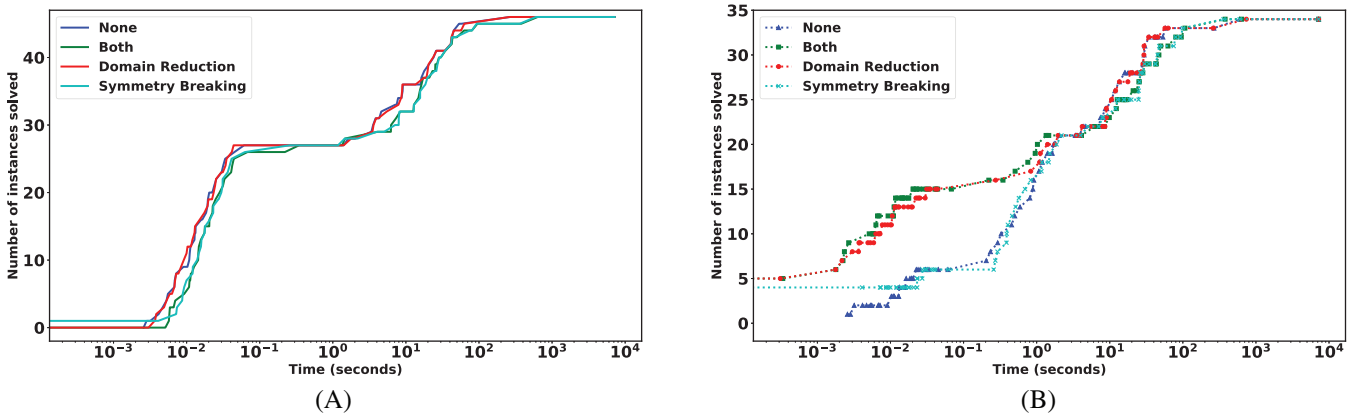


FIGURE 12 Solution times of the alternatives for structural findings. (A) Small feasible instances; (B) Small infeasible instances

5.4 | Preliminary computational study of structural findings

In this section, we present a preliminary study of the structural findings of Section 4. We implement them on $(\text{Cp}^{\text{COMBINED}})$ and test on the small random instances. In the worst case it takes 0.12 s to check for infeasibility and then apply domain reduction and symmetry breaking. For both the domain reduction and the symmetry breaking, we apply the infeasibility checks before solving. The infeasibility checks are able to prove four instances are infeasible without having to solve a mathematical program.

Solution times for $(\text{Cp}^{\text{COMBINED}})$ on small instances with domain reduction only and the symmetry breaking only, as well as both domain reduction and symmetry breaking together, and with no additions are shown in Figure 12. In total, for the set of 81 small instances, Infeasibility Check 1 was able to solve two instances and Infeasibility Check 3 was also able to solve two instances. Domain Reduction Rule 1 was applied to 12 instances. Finally, Symmetry Breaking Condition 4 was applied to 33 instances, Symmetry Breaking Condition 5 was applied to 32 instances, and Symmetry Breaking Condition 6 was applied to 37 instances. None of the instances could be deemed infeasible using Infeasibility Checks 2 and 4. We were also unable to apply Domain Reduction Rule 2, and could not use Symmetry Breaking Conditions 1, 2, or 3 for these instances.

We present the results in Table S8. We focus on small instances since we were unable to apply any of the findings other than arbitrary symmetry breaking to the medium-sized instances. We also see that the solution time with the addition of some combination of the structural strategies is decreased for 63 of the 81 instances. We believe this is because as n increases, an instance is less likely to have degree less than $2K$, since K is small with respect to n and it is less likely that two vertices will have the same neighborhoods. We observe that for feasible instances, symmetry breaking is slightly detrimental to the performance of $(\text{Cp}^{\text{COMBINED}})$ and that adding only domain reduction gives very similar results to no additions at all. However, for infeasible instances, we observe that domain reduction alone improves upon $(\text{Cp}^{\text{COMBINED}})$, and has similar performance to when both symmetry breaking and domain reduction are added to the model.

For the majority of instances there exists an addition of one or more structural finding to the model that improves the solution time as compared to $(\text{Cp}^{\text{COMBINED}})$ on its own. Thus we conclude that the structural findings are beneficial for this data set. We also see that as the instance size increases we are unable to find these key structures in the graph. This motivates the future work of incorporating these structural insights into the CP search tree for further propagation opportunities and apply them at every node of the search tree to some subset of the order.

5.5 | Preliminary computational study of valid inequalities

To provide insight into the class of valid inequalities using stable sets, (7) and (8), presented in Section 4.4, we study the impact of adding all possible valid inequalities to both $(\text{Cp}^{\text{PRANK}})$ and $(\text{Cp}^{\text{COMBINED}})$ before solving. We use the NetworkX Python package [5] to enumerate all maximal stable sets for each small instance; the total time to enumerate these sets is 2.55 s. We then add the valid inequalities (7) or (8) to the model as constraints using these stable sets and compare against the formulations without the valid inequalities. We remark that this is by no means a complete study of the impact of the valid inequalities but is meant to provide insight that even with a naive implementation there are cases where the valid inequalities are useful. The complete results table can be found in Table 9 of the Online Supplement.

Figure 13(A) shows the results for small feasible instances. We observe that adding the valid inequalities was only slightly detrimental for both $(\text{Cp}^{\text{PRANK}})$ and $(\text{Cp}^{\text{COMBINED}})$ and that the inequalities (7) have better performance than the inequalities (8). In the infeasible case, Figure 13(B), both forms of the valid inequalities increase the number of instances solved by $(\text{Cp}^{\text{PRANK}})$

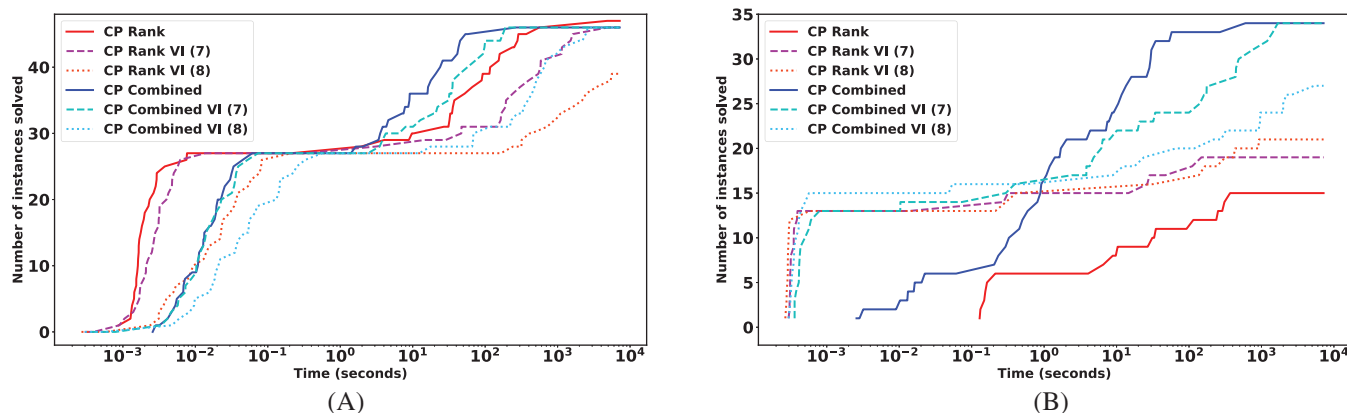


FIGURE 13 Solution times of the alternatives for valid inequalities (VI). (A) Small feasible instances; (B) Small infeasible instances

within the time limit and the speed at which they were solved. For $(CP^{COMBINED})$ on the smallest infeasible instances, the formulations with valid inequalities (7) and (8) dominate the formulation without. In fact, the formulations with valid inequalities are able to solve some of these instances with 0 choice points, which is notable. As the size of the instances grows, $(CP^{COMBINED})$ without valid inequalities has better performance. This is most likely as a result of the larger instances having more stable sets and the subsequent models having a large number of constraints as a result.

We conclude that in this preliminary implementation the valid inequalities are useful in small, low density instances and to improve (CP^{RANK}) on infeasible instances. Future work on this topic includes further analysis to select a small enough subset of the stable sets that the number of constraints will not be too large and that will have a large impact on solution times. We remark however that the valid inequalities show promise given a more suitable implementation, such as a specialized propagator for the CP formulations.

6 | CONCLUSION

We propose the first CP formulations for the CTOP and compare them against two existing IP formulations in the literature. Applying ideas from both the CP and IP literature, and by studying the structure of DMDGP orders, we introduce three classes of additions to formulations for CTOP, namely infeasibility checks, domain reduction, and symmetry breaking. We also provide the first class of valid inequalities for CTOP.

Our computational results show that our models outperform the state-of-the-art IP formulations. They also indicate that the additions to the models based on structural findings are useful for infeasible instances, but may negatively impact the amount of time it takes to solve feasible instances. Nonetheless, they give important insight into the structure of CTOP and show promise for improvement and extension.

ACKNOWLEDGEMENTS

We are grateful to Leo Liberti for introducing us to the area of distance geometry and this topic in particular. We would also like to thank J  r  my Omer for kindly providing the pseudo-protein instances similar to those used in their paper [18].

ORCID

Maira MacNeil  <https://orcid.org/0000-0002-2197-8484>

Merve Bodur  <https://orcid.org/0000-0002-9276-3755>

REFERENCES

- [1] A. Cassioli, O. G  nl  k, C. Lavor, and L. Liberti, *Discretization vertex orders in distance geometry*, *Discr. Appl. Math.* **197** (2015), 27–41.
- [2] D. S. Gon  alves and A. Mucherino, *Optimal partial discretization orders for discretizable distance geometry*, *Int. Trans. Oper. Res.* **23** (2016), 947–967.
- [3] D. Gon  alves, J. Nicolas, A. Mucherino, and C. Lavor, *Finding optimal discretization orders for molecular distance geometry by answer set programming*, in *Recent advances in computational optimization*, S. Fidanova, Ed., Springer, New York, NY, 2016, 1–15.

- [4] D. S. Gonçalves, A. Mucherino, C. Lavor, and L. Liberti, *Recent advances on the interval distance geometry problem*, J. Glob. Optim. **69** (2017), 525–545.
- [5] A. Hagberg, P. Swart, and D. S. Chult, *Exploring network structure, dynamics, and function using NetworkX*, Technical report LA-UR-08-05495; LA-UR-08-5495, Los Alamos National Lab.(LANL), Los Alamos, NM, 2008.
- [6] N. Krislock and H. Wolkowicz, *Explicit sensor network localization using semidefinite representations and facial reductions*, SIAM J. Optim. **20** (2010), 2679–2708.
- [7] C. Lavor, J. Lee, A. L.-S. John, L. Liberti, A. Mucherino, and M. Sviridenko, *Discretization orders for distance geometry problems*, Optim. Lett. **6** (2012), 783–796.
- [8] C. Lavor, L. Liberti, W. A. Lodwick, and T. M. da Costa, *An introduction to distance geometry applied to molecular geometry*, Springer, New York, NY, 2017.
- [9] C. Lavor, L. Liberti, N. Maculan, and A. Mucherino, *The discretizable molecular distance geometry problem*, Comput. Optim. Appl. **52** (2012), 115–146.
- [10] C. Lavor, L. Liberti, and A. Mucherino, *The interval branch-and-prune algorithm for the discretizable molecular distance geometry problem with inexact distances*, J. Glob. Optim. **56** (2013), 855–871.
- [11] C. Lavor, M. Souza, L. M. Carvalho, and L. Liberti, *On the polynomiality of finding KDMDGP re-orders*, Discr. Appl. Math. **267** (2019), 190–194.
- [12] C. Lecoutre, *Constraint networks: Targeting simplicity for techniques and algorithms*, John Wiley & Sons, Hoboken, NJ, 2013.
- [13] L. Liberti, C. Lavor, N. Maculan, and A. Mucherino, *Euclidean distance geometry and applications*, SIAM Rev. **56** (2014), 3–69.
- [14] A. Mucherino, *A pseudo de Bruijn graph representation for discretization orders for distance geometry*, Bioinformatics and Biomedical Engineering, Springer International Publishing, Berlin, Germany, 2015, 514–523.
- [15] A. Mucherino, *On the discretization of distance geometry: Theory, algorithms and applications*, Habilitation à diriger des recherches, IRISA, 2018.
- [16] A. Mucherino, C. Lavor, and L. Liberti, *The discretizable distance geometry problem*, Optim. Lett. **6** (2012), 1671–1686.
- [17] A. Mucherino, C. Lavor, L. Liberti, and N. Maculan, *Distance geometry: Theory, methods, and applications*, Springer Science & Business Media, Berlin, Germany, 2012.
- [18] J. Omer and D. S. Gonçalves, *An integer programming approach for the search of discretization orders in distance geometry problems*, Optim. Lett. **14** (2020), 439–452.
- [19] J. Saxe, *Embeddability of weighted graphs in K-space is strongly NP-hard*, CMU-CS-80-102, Carnegie-Mellon University, Department of Computer Science, Pittsburgh, PA, 1980.
- [20] B. M. Smith, *Dual models in constraint programming*, Technical report, School of Computing, University of Leeds, Woodhouse, Leeds, 2001.

SUPPORTING INFORMATION

Additional supporting information may be found online in the Supporting Information section at the end of the article.

How to cite this article: M. MacNeil, and M. Bodur, *Constraint programming approaches for the discretizable molecular distance geometry problem*, Networks. **79** (2022), 515–536. <https://doi.org/10.1002/net.22068>

APPENDIX A: DETAILS OF THE EXISTING IP MODELS

Prior to this work, Cassioli et al. [1] present two IP formulations for CTOP.

A.1 | The vertex-rank IP

Let x_{vr} be a binary variable, which takes value 1 if a vertex $v \in \mathcal{V}$ is receives rank $r \in [n-1]$ in the order, and 0 otherwise. Since CTOP is a satisfiability problem, we are simply looking for a feasible order; there is no objective. The so-called *vertex-rank IP* formulation is as follows:

$$(\text{IP}^{\text{VR}}) : \sum_{r \in [n-1]} x_{vr} = 1 \quad \forall v \in \mathcal{V} \quad (\text{A1a})$$

$$\sum_{v \in \mathcal{V}} x_{vr} = 1 \quad \forall r \in [n-1] \quad (\text{A1b})$$

$$\sum_{u \in \mathcal{N}(v)} \sum_{j \in [r-1]} x_{uj} \geq r x_{vr} \quad \forall v \in \mathcal{V}, r \in [1, K-1] \quad (\text{A1c})$$

$$\sum_{u \in \mathcal{N}(v)} \sum_{j \in [r-K, r-1]} x_{uj} \geq K x_{vr} \quad \forall v \in \mathcal{V}, r \in [K, n-1] \quad (\text{A1d})$$

$$x_{vr} \in \{0, 1\} \quad \forall v \in \mathcal{V}, r \in [n-1] \quad (\text{A1e})$$

Constraints (A1a) and (A1b) enforce a one-to-one assignment between the vertices and the ranks, so that each vertex appears exactly once in the order and that each rank gets exactly one vertex. Constraints (A1c) enforce that there must be an initial clique of size K , that is, that the vertices in positions $[K - 1]$ are all adjacent to their predecessors. Constraints (A1d) enforce each vertex with a rank in $[K, n - 1]$ has at least K contiguous predecessors. Finally, constraints (A1e) ensure the binary domain of the vertex-rank variables.

A.2 | The clique digraph IP and its relaxation

As stated in Key Property 2, a DMDGP order is a series of overlapping induced $(K + 1)$ -cliques in G , which cover all the vertices and share K vertices between adjacent pairs. Define a clique digraph $D = (\mathcal{O}, \mathcal{A})$, where $\mathcal{O}$ is the index set of all *ordered* cliques $\{o_j\}_{j \in \mathcal{O}}$ of size $K + 1$ in the input graph G , where $o_j = (v_1^j, v_2^j, \dots, v_{K+1}^j)$, that is, represented simply by its ordered vertices. There is an arc $(o_i, o_j) \in \mathcal{A}$ between $o_i, o_j \in \mathcal{O}$ if $v_{K+1}^i = v_1^j$, i.e., if the two cliques overlap by K vertices and differ only by the first and the last vertex, respectively. For instance, in the example given in Figure 2, there will be an arc in $\mathcal{A}$ between the vertices corresponding to the ordered 3-cliques (v_4, v_2, v_3) and (v_2, v_3, v_1) . Let ℓ_i be the last vertex of a clique $o_i \in \mathcal{O}$. In this setting a DMDGP order is described by a path (of cliques) $P = (o_1, o_2, \dots, o_{n-K})$ in D where $\{v \in o_{c_1}\} \cup \{\ell_i : i \in [2, n - K]\} = \mathcal{V}$. That is, the initial clique and the last vertices of all other cliques cover $\mathcal{V}$. For instance, the DMDGP order given in Figure 2(N) is described by the path of cliques $o_1 = (v_4, v_2, v_3)$, $o_2 = (v_2, v_3, v_1)$, $o_3 = (v_3, v_1, v_5)$ and $o_4 = (v_1, v_5, v_0)$.

Define binary variables $x_{ij} = 1$ if the arc $(i, j) \in \mathcal{A}$ is selected in the path solution P , 0 otherwise. Let the binary variable $\gamma_j = 1$ if $j \in \mathcal{O}$ is the first clique in P and $\lambda_j = 1$ if $j \in \mathcal{O}$ is the last clique in P . Define binary precedence variables $p_{uv} = 1$ if $u \in \mathcal{V}$ precedes $v \in \mathcal{V}$ in the DMDGP order. Then the *clique digraph IP* formulation is as follows:

$$(\text{IP}^{\text{CD}}) : \min \sum_{(i,j) \in \mathcal{A}} x_{ij} \quad (\text{A2a})$$

$$\text{s.t.} \sum_{j \in \mathcal{O}} \gamma_j = 1 \quad (\text{A2b})$$

$$\sum_{j \in \mathcal{O}} \lambda_j = 1 \quad (\text{A2c})$$

$$\gamma_i + \sum_{(j,i) \in \mathcal{A}} x_{ji} = \lambda_i + \sum_{(i,j) \in \mathcal{A}} x_{ij} \quad \forall i \in \mathcal{O} \quad (\text{A2d})$$

$$\sum_{(i,j) \in \mathcal{A}} x_{ij} \leq 1 \quad \forall i \in \mathcal{O} \quad (\text{A2e})$$

$$\sum_{\substack{j \in \mathcal{O} : \\ v \in o_j}} \gamma_j + \sum_{\substack{(i,j) \in \mathcal{A} : \\ o_j \setminus o_i = \{v\}}} x_{ij} = 1 \quad \forall v \in \mathcal{V} \quad (\text{A2f})$$

$$p_{uv} + p_{vu} = 1 \quad \forall v, u \in \mathcal{V} \text{ s.t. } v \neq u \quad (\text{A2g})$$

$$p_{uv} + p_{vw} + p_{wu} \leq 2 \quad \forall v, u, w \in \mathcal{V} \text{ s.t. } v \neq u \neq w \quad (\text{A2h})$$

$$p_{v_k^i, v_{k+1}^i} \geq x_{ij} \quad \forall (i, j) \in \mathcal{A}, k \in [1, K] \quad (\text{A2i})$$

$$p_{v_{K+1}^i, v} \geq x_{ij} \quad \forall (i, j) \in \mathcal{A}, v = o_j \setminus o_i \quad (\text{A2j})$$

$$p_{v_k^i, v_{k+1}^i} \geq \gamma_i + \lambda_i \quad \forall i \in \mathcal{O}, k \in [1, K] \quad (\text{A2k})$$

$$\gamma^i \in \{0, 1\} \quad \forall i \in \mathcal{O} \quad (\text{A2l})$$

$$\lambda^i \in \{0, 1\} \quad \forall i \in \mathcal{O} \quad (\text{A2m})$$

$$x_{ij} \in \{0, 1\} \quad \forall (i, j) \in \mathcal{A} \quad (\text{A2n})$$

$$p_{uv} \in \{0, 1\} \quad \forall u, v \in \mathcal{V} \quad (\text{A2o})$$

Objective (A2a) imposes that we will select the minimum number of arcs required to form the path P . Constraints (A2b) and (A2c) ensure there is exactly one initial clique and one last clique selected. Constraints (A2d) ensure that flow balance holds in the path P except at the first and last path vertices which have one arc out and one arc in respectively. These flow balance constraints also ensure a correct predecessor relationship between the cliques in P . Constraints (A2e) ensure each clique has

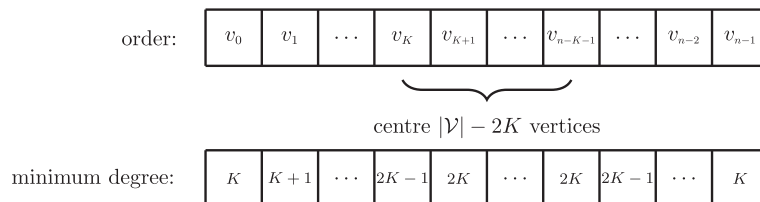


FIGURE A1 Minimum degree requirement for ranks in a discretizable molecular distance geometry problem order

at most one successor, one if it is in the path and not the last clique and none otherwise. Constraints (A2f) ensure that the cliques selected cover all the vertices in $\mathcal{V}$. Constraints (A2g) and (A2h) impose a linear order among vertex pairs and triplets. Constraints (A2i), (A2j), and (A2k)⁴ ensure that each clique is ordered. Constraints (A2i) impose that v_k^i precedes vertex v_{k+1}^i if ordered clique i has an outgoing arc in the path solution P . Constraints (A2j) ensure that if arc $(i, j) \in \mathcal{A}$ is selected in P , the vertex of j not in i , v , is preceded by all other vertices of j in i , which have been ordered by (A2i). Constraints (A2k) are similar to (A2i), except they order the vertices of the first and last clique. Finally, constraints (A2l), (A2m), (A2n), and (A2o) enforce the binary domains of all variables.

(IIP^{CD}) is disadvantaged by the potential number of vertices in the clique digraph D , as the cardinality of $\mathcal{O}$ can be quite large even for relatively sparse graphs. To reduce the number of variables in (IIP^{CD}), Cassioli et al. [1] present a relaxation of the clique digraph formulation which considers unordered cliques. The idea is to relax the ordering constraints in the formulation and to solve this relaxation as a first check for the existence of a DMDGP order. In this case, the worst-case number of vertices in D can be reduced by a factor of $(K+1)!$. Let $\mathcal{O}$ now denote the set of *unordered* cliques of size $K+1$ in G . Let binary variable $z_j = 1$ if the unordered clique $j \in \mathcal{O}$ is used in P . The unordered clique *IP relaxation* of (IIP^{CD}) is as follows:

$$(\text{IIP}^{\text{RELAX}}) : \min \sum_{(i,j) \in \mathcal{A}} x_{ij} \quad (\text{A3a})$$

$$\text{s.t. (A2b) - (A2h)} \quad (\text{A3b})$$

$$p_{uv} \geq x_{ij} \quad \forall (i,j) \in \mathcal{A}, u \in o_i, v = o_j \setminus o_i \quad (\text{A3c})$$

$$z_j \geq x_{ij} \quad \forall (i,j) \in \mathcal{A} \quad (\text{A3d})$$

$$z_i \geq \gamma_i \quad \forall i \in \mathcal{O} \quad (\text{A3e})$$

$$\sum_{i \in \mathcal{O} : v \in o_i} z_i \leq K+1 \quad \forall v \in \mathcal{V} \quad (\text{A3f})$$

$$(A2l) - (A2n) \quad (\text{A3g})$$

$$z_i \in \{0, 1\} \quad \forall i \in \mathcal{O} \quad (\text{A3h})$$

(A3a) and (A3b) are the same as in (IIP^{CD}). However, we have relaxed the clique ordering constraints, (A2i)-(A2k), so now constraints (A3c) ensure we have that if arc $(i, j) \in \mathcal{A}$ is selected in P , all $u \in o_i$ precede vertex v , the only vertex of j not in i . Constraints (A3d) ensure we have correctly linked the arc variables and clique variables to the indicator z_j , so that it is 1 if a clique is part of an arc selected in the solution path P , while the indicator for the first clique is activated through constraints (A3e). The constraints (A3f) impose that each vertex appears in at most $K+1$ cliques. These constraints are another relaxation, since to make it exact we would need to enforce that all vertices except the first and last K appear in $K+1$ cliques; however, this will require many more variables to express. Finally constraints (A3g) and (A3h) enforce the variable domains.

When a solution to ($\text{IIP}^{\text{RELAX}}$) is found, it must be verified as this solution does not necessarily yield a DMDGP order. The verification is a simple check to ensure the p solution forms a DMDGP order. The strength in this formulation is that if ($\text{IIP}^{\text{RELAX}}$) is infeasible, there is no DMDGP order for the instance.

⁴In [1], the variables for predecessors are given as w_m , only in these constraints. We believe this is a typo that the variables are in fact p_{uv} .

APPENDIX B: SELECTED PROOFS

B.1 | PROOF OF PROPOSITION 1

We claim that setting $x_{vr} = \frac{1}{n}$ for all $v \in \mathcal{V}, r \in [n-1]$ always yields a feasible solution to the LP relaxation of (IP^{VR}) . We show the proposed solution satisfies all LP constraints. For constraints (A1a), fixing $v \in \mathcal{V}$ gives

$$\sum_{r \in [n-1]} \frac{1}{n} = n \cdot \frac{1}{n} = 1.$$

Similarly, for constraints (A1b), fixing $r \in [n-1]$ gives

$$\sum_{v \in \mathcal{V}} \frac{1}{n} = |\mathcal{V}| \cdot \frac{1}{n} = n \cdot \frac{1}{n} = 1.$$

For constraints (A1c), for any $v \in \mathcal{V}$ and $r \in [1, K-1]$ we have

$$\sum_{u \in \mathcal{N}(v)} \sum_{j \in [r-1]} \frac{1}{n} \geq \sum_{j \in [r-1]} \frac{1}{n} = r \frac{1}{n},$$

where the inequality follows from $|\mathcal{N}(v)| \geq 1$ since G is connected. Finally, for constraints (A1d), for any $v \in \mathcal{V}$ and $r \in [K, n-1]$ we have

$$\sum_{u \in \mathcal{N}(v)} \sum_{j \in [r-K, r-1]} \frac{1}{n} \geq \sum_{j \in [r-K, r-1]} \frac{1}{n} = K \cdot \frac{1}{n},$$

again, due to $|\mathcal{N}(v)| \geq 1$ since G is connected.

Thus $x_{vr} = \frac{1}{n}$ satisfies all constraints and the LP relaxation is feasible.

B.2 | PROOF OF PROPOSITION 2

Proof. Minimally, the vertex in position 0 must be adjacent to K vertices since it is in a $(K+1)$ -clique, similarly for the vertex in position $n-1$. The vertex in position 1 is in two $(K+1)$ -cliques since it is in the initial clique but also is an adjacent predecessor of the vertex in position $K+1$; similarly for the vertex in position $n-2$. We extend this logic to all vertices in the order, noting that the centre $|\mathcal{V}| - 2K$ vertices in the order, that is, the ones in positions $[K, n-K-1]$, must all be in $2K$ cliques, as seen in Figure A1. This minimal case analysis gives a lower bound on the number of edges in a DMDGP order. For each vertex from position 0 to position $n-1$ we sum over the minimum degree in G to have a DMDGP order. Note that we divide by two to avoid double counting since the input graph is undirected.

$$\begin{aligned} & \frac{1}{2}(K + (K+1) + (K+2) + \cdots + 2K + \cdots + 2K + \cdots + (K+2) + (K+1) + K) \\ &= \frac{(|\mathcal{V}| - 2K)2K + 2 \sum_{i=K}^{2K-1} i}{2} \\ &= K(|\mathcal{V}| - 2K) + \sum_{i=K}^{2K-1} i \\ &= K(|\mathcal{V}| - 2K) + \frac{1}{2}K(3K-1) \\ &= \left(|\mathcal{V}| - \frac{1}{2}\right)K - \frac{1}{2}K^2. \end{aligned}$$

Thus the minimum number of edges for a DMDGP order is $\left(|\mathcal{V}| - \frac{1}{2}\right)K - \frac{1}{2}K^2$. ■

B.3 | PROOF OF PROPOSITION 3

Proof. We first show that there exists an instance for which (5) dominates (6). Consider the graph, G_1 , shown in Figure B1(A) and let $K = 3$. Let $S_1 = \{v_1, v_2, v_3, v_4, v_5\}$. Then $G_1[S_1] = G_1$ does not have a DMDGP order due to Infeasibility Check 1, for example, since v_3 cannot be in the initial clique and cannot have K contiguous predecessors.

To find the inequality (5), we calculate $\max_{v \in S_1} \delta_{S_1}^{\text{miss}}(v) = \delta_{S_1}^{\text{miss}}(v_3) = 3$ and find:

$$r_{\max} - r_{\min} \geq \max\{|S_1|, \delta_{S_1}^{\text{miss}} + K\} \quad (\text{B2a})$$

$$r_{\max} - r_{\min} \geq \max\{5, 3 + 3\} \quad (\text{B2b})$$

$$r_{\max} - r_{\min} \geq 6 \quad (\text{B2c})$$

The instance only has five vertices in total, so this single inequality is enough to prove the instance does not have a DMDGP order. The largest stable set in the graph in Figure B1(A) has cardinality two; using any of these stable sets as SS_1 makes the inequality (6)

$$r_{\max} - r_{\min} \geq (|SS_1| - 1)(K + 1) \quad (\text{B3a})$$

$$r_{\max} - r_{\min} \geq (2 - 1)(3 + 1) \quad (\text{B3b})$$

$$r_{\max} - r_{\min} \geq 4 \quad (\text{B3c})$$

which is not sufficient to prove the infeasibility of the instance. Thus (5) dominates (6) in this instance.

We now show that there exists an instance for which (6) dominates (5). Consider the graph, G_2 , shown in Figure B1b and let $K = 3$. A maximum stable set in G_2 is $SS_2 = \{u_1, u_3, u_5\}$, making the inequality (6):

$$r_{\max} - r_{\min} \geq (|SS_2| - 1)(K + 1) \quad (\text{B4a})$$

$$r_{\max} - r_{\min} \geq (3 - 1)(3 + 1) \quad (\text{B4b})$$

$$r_{\max} - r_{\min} \geq 8. \quad (\text{B4c})$$

Setting $S_2 = SS_2$, then $G[S_2]$ does not have a DMDGP order due to Infeasibility Check 1. We calculate $\max_{u \in S_2} \delta_{S_2}^{\text{miss}}(u) = 2$ and find

$$r_{\max} - r_{\min} \geq \max\{|S_2|, \delta_{S_2}^{\text{miss}} + K\} \quad (\text{B5a})$$

$$r_{\max} - r_{\min} \geq \max\{3, 2 + 3\} \quad (\text{B5b})$$

$$r_{\max} - r_{\min} \geq 5 \quad (\text{B5c})$$

Thus (6) dominates (5), and we may conclude that (5) and (6) are incomparable. ■

B.4 | PROOF OF PROPOSITION 4

Proof. Let the vertices in the wheel be indexed as in Figure 8, that is, the center vertex has index 0, while the ones in the peripheral cycle are indexed from $[1, n]$ counter clock-wise starting at an arbitrary one.

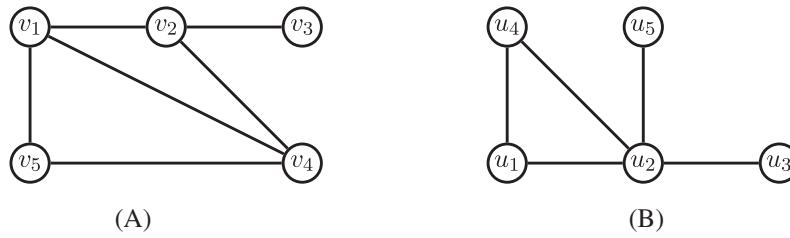
- *Case 1:* When n is odd, a maximum stable set in W_n is $\{1, 3, \dots, n - 2\}$ with size $\frac{n-1}{2}$. For the right-hand side of the inequality in Infeasibility Check 5 to hold, we need

$$\begin{aligned} \frac{n-1}{2} &\geq \frac{n}{K+1} + 1 \\ \frac{n}{2} - \frac{n}{K+1} &\geq \frac{1}{2} \\ n \left(\frac{1}{2} - \frac{1}{K+1} \right) &\geq \frac{1}{2} \\ n &\geq \frac{K+1}{K-1}. \end{aligned}$$

- *Case 2:* Similarly, when n is even, a maximum stable set in W_n is $\{1, 3, \dots, n - 3\}$ with size $\frac{n}{2}$. So we need

$$\begin{aligned} \frac{n}{2} &\geq \frac{n}{K+1} + 1 \\ n \left(\frac{1}{2} - \frac{1}{K+1} \right) &\geq 1 \\ n &\geq 2 \frac{K+1}{K-1}. \end{aligned}$$

Thus the inequalities hold. ■

FIGURE B1 Graphs used to compare valid inequalities (5) and (6). (A) G_1 ; (B) G_2

APPENDIX C: EXAMPLE

Example 10. We will demonstrate the strength of symmetry breaking on the graph in Figure C1 with $K = 2$. This instance has 12 feasible DMDGP orders:

$$\begin{array}{lll}
 (v_4, v_3, v_2, v_1, v_0, v_5) & (v_5, v_0, v_1, v_2, v_3, v_4) & (v_4, v_3, v_2, v_5, v_0, v_1) \\
 (v_1, v_0, v_5, v_2, v_3, v_4) & (v_4, v_2, v_3, v_1, v_5, v_0) & (v_0, v_5, v_1, v_3, v_2, v_4) \\
 (v_4, v_3, v_2, v_1, v_5, v_0) & (v_0, v_5, v_1, v_2, v_3, v_4) & (v_4, v_2, v_3, v_5, v_1, v_0) \\
 (v_0, v_1, v_5, v_3, v_2, v_4) & (v_4, v_3, v_2, v_5, v_1, v_0) & (v_0, v_1, v_5, v_2, v_3, v_4)
 \end{array}$$

We begin by noticing that $\mathcal{V}^{d[2,2]} = \{v_4\}$ so we fix $r_{v_4} = 0$ and eliminate half the orders, leaving the orders:

$$\begin{array}{lll}
 (v_4, v_3, v_2, v_1, v_0, v_5) & (v_4, v_2, v_3, v_1, v_5, v_0) & (v_4, v_3, v_2, v_5, v_0, v_1) \\
 (v_4, v_3, v_2, v_1, v_5, v_0) & (v_4, v_3, v_2, v_5, v_1, v_0) & (v_4, v_2, v_3, v_5, v_1, v_0)
 \end{array}$$

There are no stable sets meeting Symmetry Breaking Condition 2, and the only clique meeting Symmetry Breaking Condition 3 is $\{v_1, v_5\}$. Thus we enforce $r_{v_1} < r_{v_5}$ and eliminate another three orders, leaving three remaining orders:

$$(v_4, v_3, v_2, v_1, v_0, v_5) \quad (v_4, v_3, v_2, v_1, v_5, v_0) \quad (v_4, v_3, v_2, v_5, v_0, v_1).$$

Since we have found a clique in Symmetry Breaking Condition 3, we first examine Symmetry Breaking Condition 5. Beginning with $\mathcal{K} = \{v_1, v_5\}$ and $v = v_2$, we have

$$(\mathcal{N}(v_2) \cup \{v_2\}) \setminus (\mathcal{N}(\{v_1, v_5\}) \cup \{v_1, v_5\}) = \{v_0, v_1, v_2, v_3, v_4, v_5\} \setminus \{v_0, v_1, v_2, v_3, v_5\} = \{v_4\}$$

so $w = v_4$ and we can add the following logical constraints:

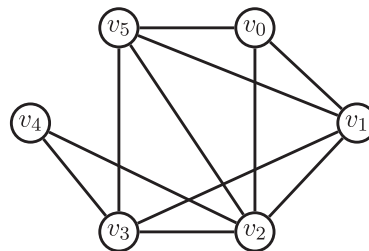
$$\begin{aligned}
 |r_{v_2} - r_{v_4}| \geq 3 &\Rightarrow r_{v_2} < r_{v_5} \\
 |r_{v_2} - r_{v_4}| \geq 3 &\Rightarrow r_{v_2} < r_{v_1}.
 \end{aligned}$$

In fact, the latter suffices since we have already added $r_{v_1} < r_{v_5}$. Unfortunately this does not remove any solutions from the pool. We now try $\mathcal{K} = \{v_0\}$ and $v = v_5$. In this case we have $w = v_3$ and add

$$|r_{v_5} - r_{v_3}| \geq 3 \Rightarrow r_{v_5} < r_{v_0},$$

which removes a further order, yielding the remaining orders:

$$(v_4, v_3, v_2, v_1, v_5, v_0) \quad (v_4, v_3, v_2, v_5, v_0, v_1).$$

FIGURE C1 A graph instance which is feasible for contiguous trilateration ordering problem with $K = 2$

Finally, we extend $\mathcal{K} = \{v_3\}$ using $v = 2$, giving $w = v_0$ and the logical constraint:

$$|r_{v_2} - r_{v_0}| \geq 3 \Rightarrow r_{v_2} < r_{v_3}$$

Thus symmetry breaking has reduced the solution space to a single DMDGP order:

$$(v_4, v_3, v_2, v_5, v_0, v_1).$$

APPENDIX D: SUMMARY OF THE PAPER

Table D1 provides a summary of the models from the literature as well as all of our proposed formulations and structural insights.

TABLE D1 Summary of contiguous trilateration ordering problem formulations and structural findings

DMDGP: Given a graph $G = (\mathcal{V}, \mathcal{E})$ and integer $K > 0$, minimally, a DMDGP order is a series of $(K + 1)$ -cliques which overlap by at least K vertices						
FORMULATIONS	Variables	Domain	Number	Constraints	Number	Comments
LITERATURE						
Vertex-rank (IP^{VR})	x_{vr}	$\{0, 1\}$	n^2	1-1 assignment	$2n$	
				clique	n^2	
Clique Digraph (IP^{CD})	$x_{o_i o_j}$	$\{0, 1\}$	$ \mathcal{O} ^2$	Select initial and last clique	2	Enumerate ordered
	γ_{o_j}	$\{0, 1\}$	$ \mathcal{O} $	Flow balance	$ \mathcal{O} $	$(K + 1)$ -cliques
	λ_{o_j}	$\{0, 1\}$	$ \mathcal{O} $	Successor	$ \mathcal{O} $	
	p_{uv}	$\{0, 1\}$	$ \mathcal{V} ^2$	Vertex covering	$ \mathcal{V} $	
				Precedence	$ \mathcal{V} ^2 + \mathcal{V} ^3$	
				Clique ordering	$(2K + 1) \times \mathcal{O} ^2$	
Clique Digraph Relax. (IP^{RELAX})	$x_{o_i o_j}$	$\{0, 1\}$	$ \mathcal{O} ^2$	Select initial and last clique	2	Enumerate $(K + 1)$ -cliques
	γ_{o_j}	$\{0, 1\}$	$ \mathcal{O} $	Flow balance	$ \mathcal{O} $	If infeasible,
	λ_{o_j}	$\{0, 1\}$	$ \mathcal{O} $	Successor	$ \mathcal{O} $	no DMDGP order
	p_{uv}	$\{0, 1\}$	n^2	Vertex covering	$ \mathcal{V} $	o.w., may or may not have
				Precedence	$ \mathcal{V} ^2 + \mathcal{V} ^3$	DMDGP order
				Relaxed clique ordering	$2 \mathcal{O} ^2 + \mathcal{O} $	
				Number times vertex in clique	$ \mathcal{V} $	
NEW						
CP Rank (CP^{RANK})	r_v	$[n - 1]$	n	AllDifferent	1	
				clique	$ \mathcal{V} ^2$	
CP Vertex ($\text{CP}^{\text{VERTEX}}$)	v_r	$[\mathcal{V} - 1]$	n	AllDifferent	1	
				clique	$ \mathcal{V} ^2$	
CP Combined ($\text{CP}^{\text{COMBINED}}$)	r_v	$[n - 1]$	n	AllDifferent and inverse	3	Combines CP Rank and
	v_r	$[\mathcal{V} - 1]$	n	clique	$2 \mathcal{V} ^2$	CP Vertex
Structural findings				SYMMETRY BREAKING		
INFEASIBILITY CHECKS				Arbitrary		
Minimum degree	$d(v) < K$			Degree K	$r_{v_1} < r_{v_2}$	
Minimum edges	$ \mathcal{E} < \left(\mathcal{V} - \frac{1}{2}\right)K - \frac{1}{2}K^2$			Stable set same neighbors	$r_{v_i} = 0$ and $r_{v_j} = n - 1$	
UB on Small Deg. Vertices	$ \mathcal{V}^{d[K, K+\delta]} > 2(\delta + 1) + 1$			Clique same neighbors	$r_{v_1} < r_{v_2} < \dots < r_{v_k}$	
LB on Large Deg. Vertices	$ \mathcal{V}^{d[2K, n-1]} \leq n - (2K + 1)$			Extended stable set	$ r_v - r_w \geq K + 1 \Rightarrow r_v < r_u$	
Max Stable Set	$SS > \frac{n}{K+1} + 1$			Extended Clique	$ r_v - r_w \geq K + 1 \Rightarrow r_v < r_u$	
DOMAIN REDUCTION				VALID INEQUALITIES		
Small Deg. Vertices	$[d(v) - K] \cup [n - 1 - (d(v) - K), n - 1]$			Vertex subset	$r_{\max} - r_{\min} \geq S $	
Neighb. Small Deg. Vertices	$[d(v^*)] \cup [n - 1 - d(v^*), n - 1]$			Improved vertex subset	$r_{\max} - r_{\min} \geq \max\{ S , \delta_S^{\text{miss}} + K\}$	
				Stable set	$r_{\max} - r_{\min} \geq (SS - 1)(K + 1)$	